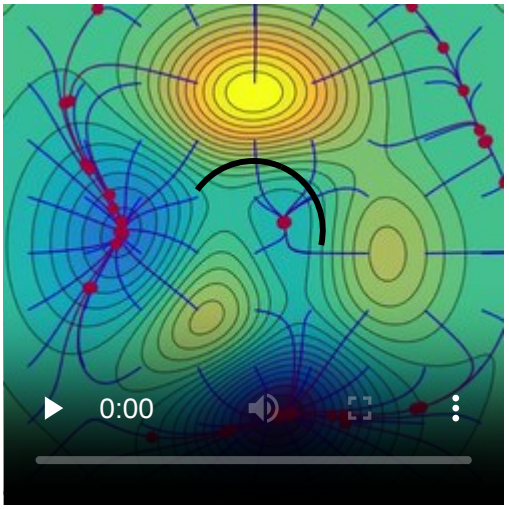
WIKIPEDIA 25
25 years of the free encyclopedia

Gradient descent

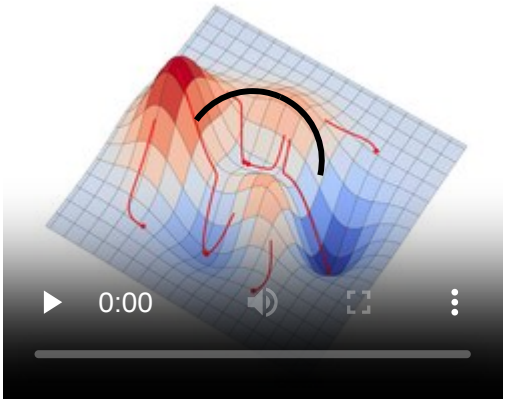
Gradient descent is a method for unconstrained mathematical optimization. It is a first-order iterative algorithm for minimizing a differentiable multivariate function.



Gradient descent in 2D

The idea is to take repeated steps in the opposite direction of the gradient (or approximate gradient) of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as *gradient ascent*. Gradient descent should not be confused with local search algorithms, although both are iterative methods for optimization.

Gradient descent is particularly useful in machine learning and artificial intelligence for minimizing the cost or loss function.[1]



Gradient descent with constant step size

Gradient descent is generally attributed to Augustin-Louis Cauchy, who first suggested it in 1847.[2] Jacques Hadamard independently proposed a similar method in 1907.[3][4] Its convergence properties for non-linear optimization problems were first studied by Haskell Curry in 1944,[5] with the method becoming increasingly well-studied and used in the following decades.[6][7]

A simple extension of gradient descent, stochastic gradient descent, serves as the most basic algorithm used for training most deep networks today.

Description

Gradient descent is based on the observation that if the multi-variable function $f(\mathbf{x})$ is defined and differentiable in a neighborhood of a point $\mathbf{a}$, then $f(\mathbf{x})$ decreases *fastest* if one goes from $\mathbf{a}$ in the direction of the negative gradient of f at $\mathbf{a}, i.e., -\nabla f(\mathbf{a})$. It follows that, if

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

for a small enough step size or learning rate $\eta \in \mathbb{R}_+$, then $f(\mathbf{a_n}) \geq f(\mathbf{a_{n+1}})$. In other words, the term $\eta \nabla f(\mathbf{a})$ is subtracted from $\mathbf{a}$ because we want to move against the gradient, toward the local minimum. With this observation in mind, one starts with a guess $\mathbf{x}_0$ for a local minimum of f, and considers the sequence $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots$ such that

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta_n \nabla f(\mathbf{x}_n),\ n \geq 0.$$

We have a monotonic sequence

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \cdots,$$

so the sequence $(\mathbf{x}_n)$ converges to the desired local minimum. Note that the value of the *step size* η is allowed to change at every iteration.

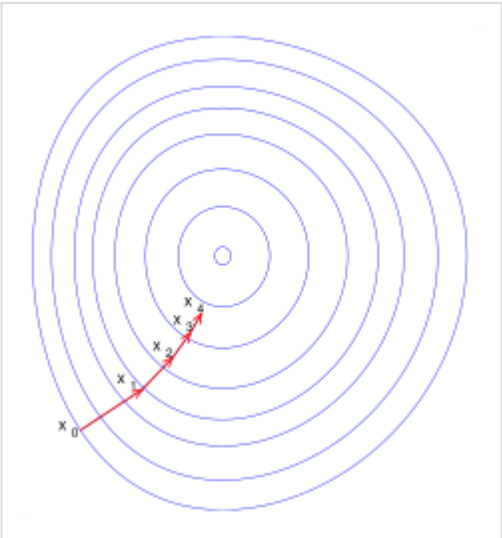


Illustration of gradient descent on a series of level sets

It is possible to guarantee the convergence to a local minimum under certain assumptions on the function f (for example, f convex and ∇f Lipschitz) and particular choices of η. Those include the sequence

$$\eta_n = \frac{\left|(\mathbf{x}_n - \mathbf{x}_{n-1})^\top [\nabla f(\mathbf{x}_n) - \nabla f(\mathbf{x}_{n-1})]\right|}{\|\nabla f(\mathbf{x}_n) - \nabla f(\mathbf{x}_{n-1})\|^2}$$

as in the Barzilai-Borwein method,[8][9] or a sequence η_n satisfying the Wolfe conditions (which can be found by using line search). When the function f is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution.

This process is illustrated in the adjacent picture. Here, f is assumed to be defined on the plane, and that its graph has a bowl shape. The blue curves are the contour lines, that is, the regions on which the value of f is constant. A red arrow originating at a point shows the direction of the negative gradient at that point. Note that the (negative) gradient at a point is orthogonal to the contour line going through that point. We see that gradient *descent* leads us to the bottom of the bowl, that is, to the point where the value of the function f is minimal.

An analogy for understanding gradient descent

Fog in the mountains

The basic intuition behind gradient descent can be illustrated by a hypothetical scenario. People are stuck in the mountains and are trying to get down (i.e., trying to find the global minimum). There is heavy fog such that visibility is extremely low. Therefore, the path down the mountain is not visible, so they must use local information to find the minimum. They can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e., downhill). If they were trying to find the top of the mountain (i.e., the maximum), then they would proceed in the direction of steepest ascent (i.e., uphill). Using this method, they would eventually find their way down the mountain or possibly get stuck in some hole (i.e., local minimum or saddle point), like a mountain lake. However, assume also that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a sophisticated instrument to measure, which the people happen to have at that moment. It takes quite some time to measure the steepness of the hill with the instrument.

Thus, they should minimize their use of the instrument if they want to get down the mountain before sunset. The difficulty then is choosing the frequency at which they should measure the steepness of the hill so as not to go off track.

In this analogy, the people represent the algorithm, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore. The steepness of the hill represents the slope of the function at that point. The instrument used to measure steepness is differentiation. The direction they choose to travel in aligns with the gradient of the function at that point. The amount of time they travel before taking another measurement is the step size.

Choosing the step size and descent direction

Since using a step size η that is too small would slow convergence, and a η too large would lead to overshoot and divergence, finding a good setting of η is an important practical problem. Philip Wolfe also advocated using "clever choices of the [descent] direction" in practice.[10] While using a direction that deviates from the steepest descent direction may seem counter-intuitive, the idea is that the smaller slope may be compensated for by being sustained over a much longer distance.

To reason about this mathematically, consider a direction $\mathbf{p}_n$ and step size η_n and consider the more general update:

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta_n \mathbf{p}_n.$$

Finding good settings of $\mathbf{p}_n$ and η_n requires some thought. First of all, we would like the update direction to point downhill. Mathematically, letting θ_n denote the angle between $-\nabla f(\mathbf{a_n})$ and $\mathbf{p}_n$, this requires that $\cos\theta_n > 0.$ To say more, we need more information about the objective function that we are optimising. Under the fairly weak assumption that f is continuously differentiable, we may prove that:[11]

$$f(\mathbf{a}_{n+1}) \leq f(\mathbf{a}_n) - \eta_n \|\nabla f(\mathbf{a}_n)\|_2 \|\mathbf{p}_n\|_2 \left(\cos\theta_n - \max_{t\in[0,1]} \frac{\|\nabla f(\mathbf{a}_n - t\eta_n\mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2}{\|\nabla f(\mathbf{a}_n)\|_2}\right) \quad \textbf{(1)}$$

This inequality implies that the amount by which we can be sure the function f is decreased depends on a trade off between the two terms in square brackets. The first term in square brackets measures the angle between the descent direction and the negative gradient. The second term measures how quickly the gradient changes along the descent direction.

In principle inequality (**1**) could be optimized over $\mathbf{p}_n$ and η_n to choose an optimal step size and direction. The problem is that evaluating the second term in square brackets requires evaluating $\nabla f(\mathbf{a}_n - t\eta_n\mathbf{p}_n)$, and extra gradient evaluations are generally expensive and undesirable. Some ways around this problem are:

- Forgo the benefits of a clever descent direction by setting $\mathbf{p}_n = \nabla f(\mathbf{a_n})$, and use line search to find a suitable step-size γ_n, such as one that satisfies the Wolfe conditions. A more economic way of choosing learning rates is backtracking line search, a method that has both good theoretical guarantees and experimental results. Note that one does not need to choose $\mathbf{p}_n$ to be the gradient; any direction that has positive inner product with the gradient will result in a reduction of the function value (for a sufficiently small value of η_n).
- Assuming that f is twice-differentiable, use its Hessian $\nabla^2 f$ to estimate $\|\nabla f(\mathbf{a}_n - t\eta_n\mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2 \approx \|t\eta_n \nabla^2 f(\mathbf{a}_n)\mathbf{p}_n\|$.Then choose $\mathbf{p}_n$ and η_n by optimising

inequality (1).

- Assuming that ∇f is Lipschitz, use its Lipschitz constant L to bound $\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2 \leq Lt\eta_n \|\mathbf{p}_n\|$. Then choose $\mathbf{p}_n$ and η_n by optimising inequality (1).
- Build a custom model of $\max_{t\in[0,1]} \frac{\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2}{\|\nabla f(\mathbf{a}_n)\|_2}$ for f. Then choose $\mathbf{p}_n$ and η_n by optimising inequality (1).
- Under stronger assumptions on the function f such as convexity, more advanced techniques may be possible.

Usually by following one of the recipes above, convergence to a local minimum can be guaranteed. When the function f is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution.

Solution of a linear system

Gradient descent can be used to solve a system of linear equations

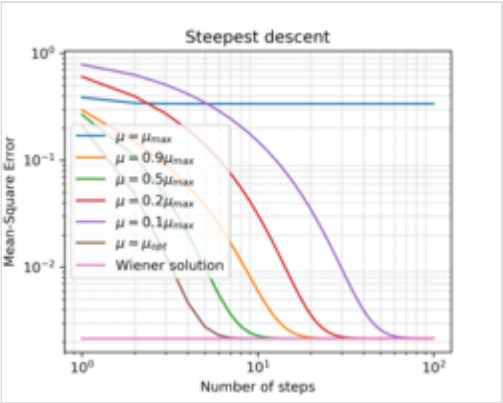
$$\mathbf{Ax} - \mathbf{b} = 0$$

reformulated as a quadratic minimization problem. If the system matrix $\mathbf{A}$ is real symmetric and positive-definite, an objective function is defined as the quadratic function, with minimization of

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Ax} - 2\mathbf{x}^\top \mathbf{b},$$

so that

$$\nabla f(\mathbf{x}) = 2(\mathbf{Ax} - \mathbf{b}).$$



The steepest descent algorithm applied to the Wiener filter[12]

For a general real matrix $\mathbf{A}$, linear least squares define

$$f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|^2.$$

In traditional linear least squares for real $\mathbf{A}$ and $\mathbf{b}$ the Euclidean norm is used, in which case

$$\nabla f(\mathbf{x}) = 2\mathbf{A}^\top(\mathbf{Ax} - \mathbf{b}).$$

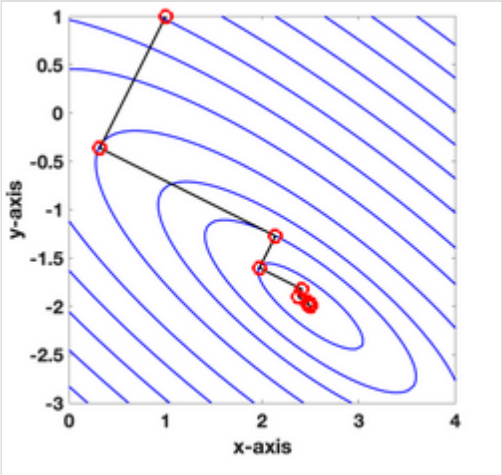
The line search minimization, finding the locally optimal step size η on every iteration, can be performed analytically for quadratic functions, and explicit formulas for the locally optimal η are known.[6][13]

For example, for real symmetric and positive-definite matrix $\mathbf{A}$, a simple algorithm can be as follows,[6]

$$
\begin{aligned}
&\text{repeat in the loop:}\\
&\qquad \mathbf{r} := \mathbf{b} - \mathbf{A}\mathbf{x}\\
&\qquad \eta := \mathbf{r}^\top \mathbf{r} / \mathbf{r}^\top \mathbf{A}\mathbf{r}\\
&\qquad \mathbf{x} := \mathbf{x} + \eta \mathbf{r}\\
&\qquad \text{if } \mathbf{r}^\top \mathbf{r} \text{ is sufficiently small, then exit loop}\\
&\text{end repeat loop}\\
&\text{return } \mathbf{x} \text{ as the result}
\end{aligned}
$$

To avoid multiplying by $\mathbf{A}$ twice per iteration, we note that $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$ implies $\mathbf{r} := \mathbf{r} - \eta \mathbf{A}\mathbf{r}$, which gives the traditional algorithm,[14]

$$
\begin{aligned}
&\mathbf{r} := \mathbf{b} - \mathbf{A}\mathbf{x}\\
&\text{repeat in the loop:}\\
&\qquad \eta := \mathbf{r}^\top \mathbf{r} / \mathbf{r}^\top \mathbf{A}\mathbf{r}\\
&\qquad \mathbf{x} := \mathbf{x} + \eta \mathbf{r}\\
&\qquad \text{if } \mathbf{r}^\top \mathbf{r} \text{ is sufficiently small, then exit loop}\\
&\qquad \mathbf{r} := \mathbf{r} - \eta \mathbf{A}\mathbf{r}\\
&\text{end repeat loop}\\
&\text{return } \mathbf{x} \text{ as the result}
\end{aligned}
$$



Convergence path of steepest descent method for A = [[2, 2], [2, 3]]

The method is rarely used for solving linear equations, with the conjugate gradient method being one of the most popular alternatives. The number of gradient descent iterations is commonly proportional to the spectral condition number $\kappa(\mathbf{A})$ of the system matrix $\mathbf{A}$ (the ratio of the maximum to minimum eigenvalues of $\mathbf{A}^\top \mathbf{A}$), while the convergence of conjugate gradient method is typically determined by a square root of the condition number, i.e., is much faster. Both methods can benefit from preconditioning, where gradient descent may require less assumptions on the preconditioner.[14]

Geometric behavior and residual orthogonality

In steepest descent applied to solving $\mathbf{A}\mathbf{x} = \mathbf{b}$, where $\mathbf{A}$ is symmetric positive-definite, the residual vectors $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$ are orthogonal across iterations:

$$\langle \mathbf{r}_{k+1}, \mathbf{r}_k \rangle = 0.$$

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When $\kappa(\mathbf{A})$ is large, this produces a characteristic zig–zag path. The poor conditioning of $\mathbf{A}$ is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation.

As shown in the image on the right, steepest descent converges slowly due to the high condition number of $\mathbf{A}$, and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.[15]

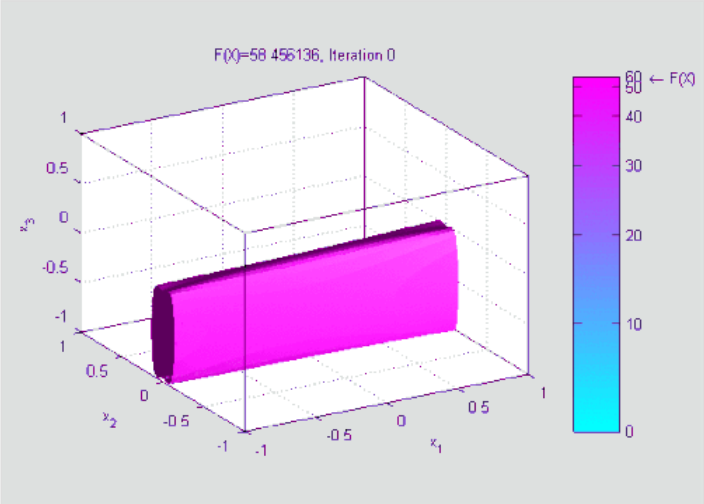
Solution of a non-linear system

Gradient descent can also be used to solve a system of nonlinear equations. Below is an example that shows how to use the gradient descent to solve for three unknown variables, x_1, x_2, and x_3. This example shows one iteration of the gradient descent.

Consider the nonlinear system of equations

$$\begin{cases} 3x_1 - \cos(x_2 x_3) - \frac{3}{2} = 0 \\ 4x_1^2 - 625x_2^2 + 2x_2 - 1 = 0 \\ \exp(-x_1 x_2) + 20x_3 + \frac{10\pi - 3}{3} = 0 \end{cases}$$

Let us introduce the associated function



An animation showing the first 83 iterations of gradient descent applied to this example. Surfaces are isosurfaces of $f(\mathbf{x}^{(n)})$ at current guess $\mathbf{x}^{(n)}$, and arrows show the direction of descent. Due to a small and constant step size, the convergence is slow.

$$G(\mathbf{x}) = \begin{bmatrix} 3x_1 - \cos(x_2 x_3) - \frac{3}{2} \\ 4x_1^2 - 625x_2^2 + 2x_2 - 1 \\ \exp(-x_1 x_2) + 20x_3 + \frac{10\pi - 3}{3} \end{bmatrix},$$

where

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

One might now define the objective function

$$\begin{aligned} f(\mathbf{x}) &= \frac{1}{2} G^{\top}(\mathbf{x}) G(\mathbf{x}) \\ &= \frac{1}{2} \left[\left(3x_1 - \cos(x_2 x_3) - \frac{3}{2} \right)^2 + \left(4x_1^2 - 625x_2^2 + 2x_2 - 1 \right)^2 + \right. \\ &\qquad \left. \left(\exp(-x_1 x_2) + 20x_3 + \frac{10\pi - 3}{3} \right)^2 \right], \end{aligned}$$

which we will attempt to minimize. As an initial guess, let us use

$$\mathbf{x}^{(0)} = \mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

We know that

$$\mathbf{x}^{(1)} = \mathbf{0} - \eta_0 \nabla f(\mathbf{0}) = \mathbf{0} - \eta_0 J_G(\mathbf{0})^\top G(\mathbf{0}),$$

where the Jacobian matrix J_G is given by

$$J_G(\mathbf{x}) = \begin{bmatrix} 3 & \sin(x_2 x_3) x_3 & \sin(x_2 x_3) x_2 \\ 8x_1 & -1250x_2 + 2 & 0 \\ -x_2 \exp(-x_1 x_2) & -x_1 \exp(-x_1 x_2) & 20 \end{bmatrix}.$$

We calculate:

$$J_G(\mathbf{0}) = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 20 \end{bmatrix}, \qquad G(\mathbf{0}) = \begin{bmatrix} -2.5 \\ -1 \\ 10.472 \end{bmatrix}.$$

Thus

$$\mathbf{x}^{(1)} = \mathbf{0} - \eta_0 \begin{bmatrix} -7.5 \\ -2 \\ 209.44 \end{bmatrix},$$

and

$$f(\mathbf{0}) = 0.5 \left((-2.5)^2 + (-1)^2 + (10.472)^2 \right) = 58.456.$$

Now, a suitable η_0 must be found such that

$$f\left(\mathbf{x}^{(1)}\right) \leq f\left(\mathbf{x}^{(0)}\right) = f(\mathbf{0}).$$

This can be done with any of a variety of line search algorithms. One might also simply guess $\eta_0 = 0.001,$ which gives

$$\mathbf{x}^{(1)} = \begin{bmatrix} 0.0075 \\ 0.002 \\ -0.20944 \end{bmatrix}.$$

Evaluating the objective function at this value, yields

$$f\left(\mathbf{x}^{(1)}\right) = 0.5 \left((-2.48)^2 + (-1.00)^2 + (6.28)^2 \right) = 23.306.$$

The decrease from $f(\mathbf{0}) = 58.456$ to the next step's value of

$$f\left(\mathbf{x}^{(1)}\right) = 23.306$$

is a sizable decrease in the objective function. Further steps would reduce its value further until an approximate solution to the system was found.

Comments

Gradient descent works in spaces of any number of dimensions, even in infinite-dimensional ones. In the latter case, the search space is typically a function space, and one calculates the Fréchet derivative of the functional to be minimized to determine the descent direction.[7]

That gradient descent works in any number of dimensions (finite number at least) can be seen as a consequence of the Cauchy–Schwarz inequality, i.e. the magnitude of the inner (dot) product of two vectors of any dimension is maximized when they are colinear. In the case of gradient descent, that would be when the vector of independent variable adjustments is proportional to the gradient vector of partial derivatives.

The gradient descent can take many iterations to compute a local minimum with a required accuracy, if the curvature in different directions is very different for the given function. For such functions, preconditioning, which changes the geometry of the space to shape the function level sets like concentric circles, cures the slow convergence. Constructing and applying preconditioning can be computationally expensive, however.

The gradient descent can be modified via momentums[16] (Nesterov, Polyak,[17] and Frank–Wolfe[18]) and heavy-ball parameters (exponential moving averages[19] and positive-negative momentum[20]). The main examples of such optimizers are Adam, DiffGrad, Yogi, AdaBelief, etc.

Methods based on Newton's method and inversion of the Hessian using conjugate gradient techniques can be better alternatives.[21][22] Generally, such methods converge in fewer iterations, but the cost of each iteration is higher. An example is the BFGS method which consists in calculating on every step a matrix by which the gradient vector is multiplied to go into a "better" direction, combined with a more sophisticated line search algorithm, to find the "best" value of $\boldsymbol{\eta}$. For extremely large problems, where the computer-memory issues dominate, a limited-memory method such as L-BFGS should be used instead of BFGS or the steepest descent.

While it is sometimes possible to substitute gradient descent for a local search algorithm, gradient descent is not in the same family: although it is an iterative method for local optimization, it relies on an objective function's gradient rather than an explicit exploration of a solution space.

Gradient descent can be viewed as applying Euler's method for solving ordinary differential equations $x'(t) = -\nabla f(x(t))$ to a gradient flow. In turn, this equation may be derived as an optimal controller[23] for the control system $x'(t) = u(t)$ with $u(t)$ given in feedback form $u(t) = -\nabla f(x(t))$.

Modifications

Gradient descent can converge to a local minimum and slow down in a neighborhood of a saddle point. Even for unconstrained quadratic minimization, gradient descent develops a zig–zag pattern of subsequent iterates as iterations progress, resulting in slow convergence. Multiple modifications of gradient descent have been proposed to address these deficiencies.

Fast gradient methods

Yurii Nesterov has proposed[24] a simple modification that enables faster convergence for convex problems and has been since further generalized. For unconstrained smooth problems, the method is called the fast gradient method (FGM) or the accelerated gradient method (AGM). Specifically, if the differentiable function f is convex and ∇f is Lipschitz, and it is not assumed that f is strongly convex, then the error in the objective value generated at each step k by the gradient descent method will be bounded by $\mathcal{O}\left(k^{-1}\right)$. Using the Nesterov acceleration technique, the error decreases at $\mathcal{O}\left(k^{-2}\right)$.[25][26] It is known that the rate $\mathcal{O}\left(k^{-2}\right)$ for the decrease of the cost function is optimal for first-order optimization methods. Nevertheless, there is the opportunity to improve the algorithm by reducing the constant factor. The optimized gradient method (OGM)[27] reduces that constant by a factor of two and is an optimal first-order method for large-scale problems.[28]

For constrained or non-smooth problems, Nesterov's FGM is called the fast proximal gradient method (FPGM), an acceleration of the proximal gradient method.

Momentum or *heavy ball* method

Trying to break the zig-zag pattern of gradient descent, the *momentum or heavy ball method* uses a momentum term in analogy to a heavy ball sliding on the surface of values of the function being minimized,[6] or to mass movement in Newtonian dynamics through a viscous medium in a conservative force field.[29] Gradient descent with momentum remembers the solution update at each iteration, and determines the next update as a linear combination of the gradient and the previous update. For unconstrained quadratic minimization, a theoretical convergence rate bound of the heavy ball method is asymptotically the same as that for the optimal conjugate gradient method.[6]

This technique is used in stochastic gradient descent and as an extension to the backpropagation algorithms used to train artificial neural networks.[30][31] In the direction of updating, stochastic gradient descent adds a stochastic property. The weights can be used to calculate the derivatives.

Extensions

Gradient descent can be extended to handle constraints by including a projection onto the set of constraints. This method is only feasible when the projection is efficiently computable on a computer. Under suitable assumptions, this method converges. This method is a specific case of the forward–backward algorithm for monotone inclusions (which includes convex programming and variational inequalities).[32]

Gradient descent is a special case of mirror descent using the squared Euclidean distance as the given Bregman divergence.[33]

Theoretical properties

The properties of gradient descent depend on the properties of the objective function and the variant of gradient descent used (for example, if a line search step is used). The assumptions made affect the convergence rate, and other properties, that can be proven for gradient descent.[34] For example, if the objective is assumed to be strongly convex and lipschitz smooth, then gradient descent converges linearly with a fixed step size.[1] Looser assumptions lead to either weaker convergence guarantees or require a more sophisticated step size selection.[34]

Examples

- Yang–Mills flow
- Yang–Mills–Higgs flow
- Seiberg–Witten flow

See also

- Backtracking line search
- Conjugate gradient method
- Stochastic gradient descent
- Rprop
- Delta rule
- Wolfe conditions
- Preconditioning
- Broyden–Fletcher–Goldfarb–Shanno algorithm
- Davidon–Fletcher–Powell formula
- Nelder–Mead method
- Gauss–Newton algorithm
- Hill climbing
- Quantum annealing
- CLS (continuous local search)
- Neuroevolution

References

1. Boyd, Stephen; Vandenberghe, Lieven (2004-03-08). *Convex Optimization*. Cambridge University Press. doi:10.1017/cbo9780511804441 (https://doi.org/10.1017%2Fcbo9780511804441). ISBN 978-0-521-83378-3.
2. Lemaréchal, C. (1 January 2012). "Cauchy and the gradient method" (https://web.archive.org/web/20181229073335/https://www.math.uni-bielefeld.de/documenta/vol-ismp/40_lemarechal-claude.pdf) (PDF). In Grötschel, M. (ed.). *Optimization Stories*. Documenta Mathematica Series. Vol. 6 (1st ed.). EMS Press. pp. 251–254. doi:10.4171/dms/6/27 (https://doi.org/10.4171%2Fdms%2F6%2F27). ISBN 978-3-936609-58-5. Archived from the original (https://www.math.uni-bielefeld.de/documenta/vol-ismp/40_lemarechal-claude.pdf) (PDF) on 2018-12-29. Retrieved 2020-01-26.
3. Hadamard, Jacques (1908). "Mémoire sur le problème d'analyse relatif à l'équilibre des plaques élastiques encastrées". *Mémoires présentés par divers savants éstrangers à l'Académie des Sciences de l'Institut de France*. **33**.

4. Courant, R. (1943). "Variational methods for the solution of problems of equilibrium and vibrations" (https://doi.org/10.1090%2FS0002-9904-1943-07818-4). *Bulletin of the American Mathematical Society*. **49** (1): 1–23. doi:10.1090/S0002-9904-1943-07818-4 (https://doi.org/10.1090%2FS0002-9904-1943-07818-4).
5. Curry, Haskell B. (1944). "The Method of Steepest Descent for Non-linear Minimization Problems" (https://doi.org/10.1090%2Fqam%2F10667). *Quart. Appl. Math*. **2** (3): 258–261. doi:10.1090/qam/10667 (https://doi.org/10.1090%2Fqam%2F10667).
6. Polyak, Boris (1987). *Introduction to Optimization* (https://www.researchgate.net/publication/342978480).
7. Akilov, G. P.; Kantorovich, L. V. (1982). *Functional Analysis* (2nd ed.). Pergamon Press. ISBN 0-08-023036-9.
8. Barzilai, Jonathan; Borwein, Jonathan M. (1988). "Two-Point Step Size Gradient Methods". *IMA Journal of Numerical Analysis*. **8** (1): 141–148. doi:10.1093/imanum/8.1.141 (https://doi.org/10.1093%2Fimanum%2F8.1.141).
9. Fletcher, R. (2005). "On the Barzilai–Borwein Method". In Qi, L.; Teo, K.; Yang, X. (eds.). *Optimization and Control with Applications*. Applied Optimization. Vol. 96. Boston: Springer. pp. 235–256. ISBN 0-387-24254-6.
10. Wolfe, Philip (April 1969). "Convergence Conditions for Ascent Methods". *SIAM Review*. **11** (2): 226–235. doi:10.1137/1011036 (https://doi.org/10.1137%2F1011036).
11. Bernstein, Jeremy; Vahdat, Arash; Yue, Yisong; Liu, Ming-Yu (2020-06-12). "On the distance between two neural networks and the stability of learning". arXiv:2002.03432 (https://arxiv.org/abs/2002.03432) [cs.LG (https://arxiv.org/archive/cs.LG)].
12. Haykin, Simon S. Adaptive filter theory. Pearson Education India, 2008. - p. 108-142, 217-242
13. Saad, Yousef (2003). *Iterative methods for sparse linear systems* (https://archive.org/details/iterativemethods0000saad/page/195) (2nd ed.). Philadelphia, Pa.: Society for Industrial and Applied Mathematics. pp. 195 (https://archive.org/details/iterativemethods0000saad/page/195). ISBN 978-0-89871-534-7.
14. Bouwmeester, Henricus; Dougherty, Andrew; Knyazev, Andrew V. (2015). "Nonsymmetric Preconditioning for Conjugate Gradient and Steepest Descent Methods" (https://doi.org/10.1016%2Fj.procs.2015.05.241). *Procedia Computer Science*. **51**: 276–285. arXiv:1212.6680 (https://arxiv.org/abs/1212.6680). doi:10.1016/j.procs.2015.05.241 (https://doi.org/10.1016%2Fj.procs.2015.05.241).
15. Holmes, M. (2023). *Introduction to Scientific Computing and Data Analysis, 2nd Ed*. Springer. ISBN 978-3-031-22429-4.
16. Abdulkadirov, Ruslan; Lyakhov, Pavel; Nagornov, Nikolay (January 2023). "Survey of Optimization Algorithms in Modern Neural Networks" (https://doi.org/10.3390%2Fmath11112466). *Mathematics*. **11** (11): 2466. doi:10.3390/math11112466 (https://doi.org/10.3390%2Fmath11112466). ISSN 2227-7390 (https://search.worldcat.org/issn/2227-7390).
17. Diakonikolas, Jelena; Jordan, Michael I. (January 2021). "Generalized Momentum-Based Methods: A Hamiltonian Perspective" (https://epubs.siam.org/doi/10.1137/20M1322716). *SIAM Journal on Optimization*. **31** (1): 915–944. arXiv:1906.00436 (https://arxiv.org/abs/1906.00436). doi:10.1137/20M1322716 (https://doi.org/10.1137%2F20M1322716). ISSN 1052-6234 (https://search.worldcat.org/issn/1052-6234).
18. Meyer, Gerard G. L. (November 1974). "Accelerated Frank–Wolfe Algorithms" (http://epubs.siam.org/doi/10.1137/0312050). *SIAM Journal on Control*. **12** (4): 655–663. doi:10.1137/0312050 (https://doi.org/10.1137%2F0312050). ISSN 0036-1402 (https://search.worldcat.org/issn/0036-1402).
19. Kingma, Diederik P.; Ba, Jimmy (2017-01-29), *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980 (https://arxiv.org/abs/1412.6980)

20. Xie, Zeke; Yuan, Li; Zhu, Zhanxing; Sugiyama, Masashi (2021-07-01). "Positive-Negative Momentum: Manipulating Stochastic Gradient Noise to Improve Generalization" (https://proceedings.mlr.press/v139/xie21h.html). *Proceedings of the 38th International Conference on Machine Learning*. PMLR: 11448–11458. arXiv:2103.17182 (https://arxiv.org/abs/2103.17182).
21. Press, W. H.; Teukolsky, S. A.; Vetterling, W. T.; Flannery, B. P. (1992). *Numerical Recipes in C: The Art of Scientific Computing* (https://archive.org/details/numericalrecipes00pres_0) (2nd ed.). New York: Cambridge University Press. ISBN 0-521-43108-5.
22. Strutz, T. (2016). *Data Fitting and Uncertainty: A Practical Introduction to Weighted Least Squares and Beyond* (2nd ed.). Springer Vieweg. ISBN 978-3-658-11455-8.
23. Ross, I.M. (July 2019). "An optimal control theory for nonlinear optimization" (https://doi.org/10.1016%2Fj.cam.2018.12.044). *Journal of Computational and Applied Mathematics*. **354**: 39–51. doi:10.1016/j.cam.2018.12.044 (https://doi.org/10.1016%2Fj.cam.2018.12.044). S2CID 127649426 (https://api.semanticscholar.org/CorpusID:127649426).
24. Nesterov, Yurii (2004). *Introductory Lectures on Convex Optimization: A Basic Course*. Springer. ISBN 1-4020-7553-7.
25. Vandenberghe, Lieven (2019). "Fast Gradient Methods" (https://www.seas.ucla.edu/~vandenbe/236C/lectures/fgrad.pdf) (PDF). *Lecture notes for EE236C at UCLA*.
26. Walkington, Noel J. (2023). "Nesterov's Method for Convex Optimization" (https://epubs.siam.org/doi/10.1137/21M1390037). *SIAM Review*. **65** (2): 539–562. doi:10.1137/21M1390037 (https://doi.org/10.1137%2F21M1390037). ISSN 0036-1445 (https://search.worldcat.org/issn/0036-1445).
27. Kim, D.; Fessler, J. A. (2016). "Optimized First-order Methods for Smooth Convex Minimization" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5067109). *Mathematical Programming*. **151** (1–2): 81–107. arXiv:1406.5468 (https://arxiv.org/abs/1406.5468). doi:10.1007/s10107-015-0949-3 (https://doi.org/10.1007%2Fs10107-015-0949-3). PMC 5067109 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5067109). PMID 27765996 (https://pubmed.ncbi.nlm.nih.gov/27765996). S2CID 207055414 (https://api.semanticscholar.org/CorpusID:207055414).
28. Drori, Yoel (2017). "The Exact Information-based Complexity of Smooth Convex Minimization". *Journal of Complexity*. **39**: 1–16. arXiv:1606.01424 (https://arxiv.org/abs/1606.01424). doi:10.1016/j.jco.2016.11.001 (https://doi.org/10.1016%2Fj.jco.2016.11.001). S2CID 205861966 (https://api.semanticscholar.org/CorpusID:205861966).
29. Qian, Ning (January 1999). "On the momentum term in gradient descent learning algorithms". *Neural Networks*. **12** (1): 145–151. CiteSeerX 10.1.1.57.5612 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.57.5612). doi:10.1016/S0893-6080(98)00116-6 (https://doi.org/10.1016%2FS0893-6080%2898%2900116-6). PMID 12662723 (https://pubmed.ncbi.nlm.nih.gov/12662723). S2CID 2783597 (https://api.semanticscholar.org/CorpusID:2783597).
30. "Momentum and Learning Rate Adaptation" (http://www.willamette.edu/~gorr/classes/cs449/momrate.html). Willamette University. Retrieved 17 October 2014.
31. Geoffrey Hinton; Nitish Srivastava; Kevin Swersky. "The momentum method" (https://www.coursera.org/lecture/neural-networks/the-momentum-method-Oya9a). *Coursera*. Retrieved 2 October 2018. Part of a lecture series for the Coursera online course Neural Networks for Machine Learning (https://www.coursera.org/learn/neural-networks) Archived (https://web.archive.org/web/20161231174321/https://www.coursera.org/learn/neural-networks) 2016-12-31 at the Wayback Machine.
32. Combettes, P. L.; Pesquet, J.-C. (2011). "Proximal splitting methods in signal processing". In Bauschke, H. H.; Burachik, R. S.; Combettes, P. L.; Elser, V.; Luke, D. R.; Wolkowicz, H. (eds.). *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*. New York: Springer. pp. 185–212. arXiv:0912.3522 (https://arxiv.org/abs/0912.3522). ISBN 978-1-4419-9568-1.

33. "Mirror descent algorithm" (https://tlienart.github.io/posts/2018/10/27-mirror-descent-algorithm/).
34. Bubeck, Sébastien (2015). "Convex Optimization: Algorithms and Complexity". arXiv:1405.4980 (https://arxiv.org/abs/1405.4980) [math.OC (https://arxiv.org/archive/math.OC)].

Further reading

- Boyd, Stephen; Vandenberghe, Lieven (2004). "Unconstrained Minimization" (https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf#page=471) (PDF). *Convex Optimization*. New York: Cambridge University Press. pp. 457–520. ISBN 0-521-83378-7.
- Chong, Edwin K. P.; Żak, Stanislaw H. (2013). "Gradient Methods" (https://books.google.com/books?id=iD5s0iKXHP8C&pg=PA131). *An Introduction to Optimization* (Fourth ed.). Hoboken: Wiley. pp. 131–160. ISBN 978-1-118-27901-4.
- Himmelblau, David M. (1972). "Unconstrained Minimization Procedures Using Derivatives". *Applied Nonlinear Programming*. New York: McGraw-Hill. pp. 63–132. ISBN 0-07-028921-2.

External links

- Using gradient descent in C++, Boost, Ublas for linear regression (https://codingplayground.blogspot.it/2013/05/learning-linear-regression-with.html)
- Series of Khan Academy videos discusses gradient ascent (https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/gradient-and-directional-derivatives/v/gradient)
- Online book teaching gradient descent in deep neural network context (http://neuralnetworksanddeeplearning.com/chap1.html#learning_with_gradient_descent)
- Archived at Ghostarchive (https://ghostarchive.org/varchive/youtube/20211211/IHZwWFHWa-w) and the Wayback Machine (https://web.archive.org/web/20171016173155/https://www.youtube.com/watch?v=IHZwWFHWa-w): "Gradient Descent, How Neural Networks Learn" (https://www.youtube.com/watch?v=IHZwWFHWa-w&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=2). *3Blue1Brown*. October 16, 2017 – via YouTube.
- Garrigos, Guillaume; Gower, Robert M. (2023). "Handbook of Convergence Theorems for (Stochastic) Gradient Methods". arXiv:2301.11235 (https://arxiv.org/abs/2301.11235) [math.OC (https://arxiv.org/archive/math.OC)].

Retrieved from "https://en.wikipedia.org/w/index.php?title=Gradient_descent&oldid=1351709288"